Laravel

Vida y Milagros

Pepe Devesa

ies SERPIS | Departament Informàtica

Continguts

1 Pròleg **3**
1.1 Ejemplos de frameworks PHP . . . 3
1.2 Software necesario . . . 4

2 Laravel **4**
2.1 Instalando composer . . . 4
2.1.1 Linux . . . 4
2.1.2 Comprobación de la instalación . . . 5
2.2 Actualizando Laravel . . . 6

3 Crear proyectos Laravel **6**
3.1 Usando el comando laravel . . . 7
3.2 Crear proyectos Laravel usando versiones anteriores . . . 7
3.3 Puesta en marcha en carpeta predefinida de XAMPP . . . 8
3.4 El comando `artisan` . . . 8
3.5 Laravel y Visual Studio Code . . . 9
3.6 Estructura de un proyecto Laravel . . . 10
3.7 Configuración general del proyecto. . . . 11
3.8 Permisos en carpetas del proyecto . . . 12

4 Puesta en marcha con el comando artisan **12**

1 Pròleg

Un framework es una herramienta que proporciona una serie de módulos que ayudan a organizar y desarrollar un producto software. En el caso concreto de los frameworks PHP, la mayoría de ellos proporcionan una serie de comandos o herramientas para crear proyectos con una estructura determinada (normalmente, siguiendo el patrón MVC que veremos después), de forma que ya dan una base de trabajo hecha, y facilidades para poder crear el modelo de datos, la conexión a la base de datos, las rutas de las diferentes secciones de la aplicación, etc.

1.1 Ejemplos de frameworks PHP

Actualmente existe una gran variedad de frameworks PHP que elegir para desarrollar nuestras aplicaciones. Algunos de los más populares son:

- `Laravel`, un framework relativamente reciente (fue creado en 2011), y que ha ganado bastante popularidad en los últimos años. Su filosofía es el poder desarrollar proyectos de forma elegante y simple. Cuenta con una amplia comunidad de soporte detrás, y se le augura un futuro bastante consolidado.
- `Symfony`, creado en 2005, cuenta con más camino hecho que Laravel, y una estructura más consolidada. En sus primeras versiones se presentaba como un framework más monolítico (se instalaban demasiados módulos que luego no necesitábamos), pero recientemente ha adaptado su estructura para hacerla más modular. De hecho, podríamos considerar Symfony como un metaframework, es decir, un framework que, a su vez, sirve para desarrollar otros frameworks. Prueba de ello es que, por ejemplo, Laravel utiliza Symfony como base para ampliar esas funcionalidades.
- `CodeIgniter`, un framework más ligero que los anteriores, pero también con un amplio grupo de seguidores y desarrolladores. Fue creado en 2006 y, aunque ha sufrido una etapa de abandono, ha vuelto a coger fuerza en los últimos años, quizá debido a su simplicidad de uso. CakePHP, creado en 2005, es otro framework similar a CodeIgniter en cuanto a simplicidad y facilidad de uso, aunque con menor popularidad.
- `Zend`, creado en 2006, es otro framework bastante popular, aunque quizá con menor visibilidad que los anteriores hoy en día, a la altura de CakePHP.
- `Phalcon`, otro framework de reciente creación (2012), con una potente capacidad de procesamiento de páginas PHP, y la posibilidad de trabajar como microframework (más ligero, para ofrecer funcionalidades muy específicas) o como framework completo. De hecho, muchos frameworks más antiguos también han incorporado recientemente la posibilidad de ejecutarlos como microframeworks.

- … etc.

1.2 Software necesario

A la hora de trabajar con Laravel, necesitamos tener previamente instalados en nuestro sistema una serie de recursos software, como son:

1. Un IDE (entorno de desarrollo) con el que editar el código de nuestros proyectos. Emplearemos `Visual Studio Code` en estos apuntes, aunque existen otras alternativas similares, como PHPStorm, Sublime Text, Atom, etc.
2. Un servidor web que soporte PHP. En nuestro caso, utilizaremos `xampp`.
3. Un servidor de bases de datos en el que almacenar la información de nuestras aplicaciones. Emplearemos un servidor MariaDB/MySQL, `phpmyadmin`.
4. PHP actualizado a una versión compatible con la versión de Laravel que vayamos a utilizar. Por ejemplo, para Laravel versión 9 se necesita una versión de PHP 8.0 o posterior.

2 Laravel

Para trabajar con Laravel, será necesario instalar el gestor de paquetes **composer** mediante el que podremos tanto crear proyectos Laravel como gestionar las dependencias de otros módulos en un proyecto.

2.1 Instalando composer

Ésta es una herramienta muy habitual en ecosistemas PHP, y su labor es similar a la que desempeña el gestor NPM para aplicaciones JavaScript: gestionar las dependencias de un determinado proyecto, descargando, actualizando o desinstalando los paquetes necesarios. En este caso, lo utilizaremos para descargar e instalar el propio framework Laravel.

2.1.1 Linux

Para instalar composer en Linux y Mac OSX, debemos descargar el archivo `composer.phar` de la web oficial y copiarlo renombrado a composer desde donde lo hayamos descargado a alguna carpeta que forme parte del PATH del sistema, y activarlo como ejecutable. Por ejemplo:

```
mv composer.phar composer
sudo mv composer /usr/local/bin/composer
sudo chmod +x /usr/local/bin/composer
```

Como último paso, y ya que Composer utiliza el ejecutable de PHP, necesitamos que dicho ejecutable esté también en el PATH del sistema.

```
echo "export PATH=$PATH:/opt/lampp/bin" >> ~/.bashrc
source ~/.bashrc
```

2.1.2 Comprobación de la instalación

Una vez instalado, podemos ejecutar el comando `composer` sin más en un terminal, y comprobar que muestra una salida con las opciones que ofrece:

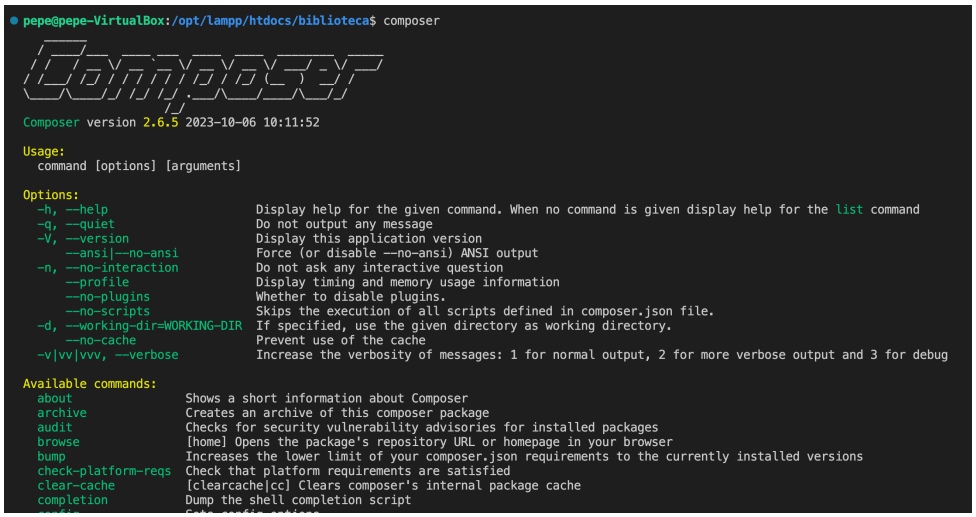


Figure 1: composer

Instalando Laravel

A través de la herramienta `composer` se pueden crear directamente proyectos Laravel, como veremos en el curso. Sin embargo, la sintaxis del comando de creación es algo larga, si la comparamos con el instalador de Laravel, por lo que vamos a instalarlo también. Para hacerlo, usamos la propia herramienta composer , con este comando:

```
composer global require laravel/installer
```

En el caso de Windows, este comando ya deja el instalador laravel listo para poderse ejecutar desde terminal. Para Linux, deberemos añadirlo al PATH del sistema (en realidad, añadimos la carpeta con

las utilidades que composer instala de forma global al sistema).

```
echo "export PATH=$PATH:$HOME/.config/composer/vendor/bin" >> ~/.bashrc
source ~/.bashrc
```

2.2 Actualizando Laravel

En general, el comando laravel que se instala se encargará de crear proyectos empleando la última versión de Laravel que haya disponible. Así, si por ejemplo lo instalamos cuando aún existía la versión 7 de Laravel pero se publica la versión 8, automáticamente el comando laravel nos permitirá crear proyectos de la versión 8 desde ese punto. Sin embargo, es posible que con el tiempo se requiera actualizar el instalador para que los proyectos con nuevas versiones se sigan creando sin problemas. Para poder actualizar a la versión más reciente de Laravel, tenemos dos opciones, aunque es cierto que ninguna de ellas está recogida en la documentación oficial de Laravel, y lo que aquí se menciona se basa en recomendaciones de webs externas a Laravel. La primera forma de actualizar es utilizar el comando de actualización:

```
composer global update laravel/installer
```

Sin embargo, esta opción puede no ser suficiente si el cambio es demasiado brusco (por ejemplo, pasar de Laravel 5 a Laravel 7), ya que algunas dependencias que también haya instaladas harían inviable el cambio. En este caso, podemos optar por quitar la versión instalada por completo, e instalar la reciente:

```
composer global remove laravel/installer
composer global require laravel/installer
```

Version actual:

```
Laravel Framework 9.43.0
```

Ahora que ya tenemos todo el sistema preparado para desarrollar proyectos Laravel, veamos los primeros pasos que debemos dar para crear estos proyectos.

3 Crear proyectos Laravel

Para crear proyectos Laravel, tenemos dos alternativas:

- Emplear el comando `laravel` que hemos instalado en sesiones anteriores mediante *composer*

- Utilizar el propio comando 'composer" para crear el proyecto. Esta opción será la recomendada si queremos crear proyectos con versiones de Laravel que no sean la última, o si por algún motivo no funciona la opción anterior.

3.1 Usando el comando laravel

Si empleamos el comando `laravel` para crear proyectos (asumiendo que ya lo tendremos instalado de la sesión de software necesario), nos deberemos ubicar en la carpeta donde queramos crear el proyecto y escribir este comando:

```
laravel new nombre_proyecto
```

Por ejemplo, para las pruebas que iremos construyendo poco a poco en las siguientes sesiones, vamos a crear una web de libros, por lo que, en la carpeta donde queramos tener este proyecto, comenzamos escribiendo este comando:

```
laravel new biblioteca
```

Esto creará un proyecto `biblioteca` en una subcarpeta con el mismo nombre. ## Usando el comando composer Alternativamente, también se puede emplear la herramienta `composer` para crear el proyecto, usando la siguiente sintaxis (también desde la carpeta donde queramos ubicar el proyecto):

```
composer create-project --prefer-dist laravel/laravel nombre_proyecto
```

En nuestro caso, para el ejemplo de la biblioteca que vamos a ir desarrollando, escribiremos este comando:

```
composer create-project --prefer-dist laravel/laravel biblioteca
```

Del mismo modo que en el caso anterior, se creará una carpeta `biblioteca` con el contenido inicial del proyecto dentro, empleando la última versión de Laravel disponible.

3.2 Crear proyectos Laravel usando versiones anteriores

Al crear un proyecto con el comando `laravel new`, se creará con la última versión disponible de Laravel. En el caso de que necesitemos crear un proyecto Laravel que no utilice la última versión, sino alguna anterior, necesitamos utilizar la herramienta composer para especificar el número de versión de Laravel que queremos utilizar. Por ejemplo, este comando crea un proyecto llamado "prueba" utilizando Laravel 7:

```
composer create-project --prefer-dist laravel/laravel prueba 7.x
```

3.3 Puesta en marcha en carpeta predefinida de XAMPP

Si nuestro proyecto se llama `biblioteca` y queremos moverlo a esta carpeta en Linux, escribimos este comando desde la carpeta que contiene nuestro proyecto:

```
sudo mv biblioteca /opt/lampp/htdocs
```

A continuación debemos poner en marcha el servidor Apache (empleando el manager de XAMPP), y luego acceder a la URL. Obtendremos la misma página de inicio que en el caso anterior.

3.4 El comando `artisan`

Cuando se crea un proyecto Laravel, se instala una herramienta llamada `artisan` en la raíz del proyecto. Es una interfaz de línea de comandos (CLI, Command Line Interface), que proporciona una serie de opciones adicionales que nos vendrán bien en nuestra gestión de proyectos Laravel para, por ejemplo, crear controladores, migrar datos a una base de datos, etc. Para comprobar que está instalada y las opciones que ofrece, podemos escribir el siguiente comando en un terminal desde la carpeta del proyecto que hayamos creado:

```
php artisan list
```

```
pepe@pepe-VirtualBox:/opt/lampp/htdocs/biblioteca$ php artisan list
Laravel Framework 9.43.0

Usage:
  command [options] [arguments]

Options:
  -h, --help            Display help for the given command. When no
  -q, --quiet           Do not output any message
  -V, --version         Display this application version
      --ansi|--no-ansi  Force (or disable --no-ansi) ANSI output
  -n, --no-interaction  Do not ask any interactive question
      --env[=ENV]       The environment the command should run under
  -v|vv|vvv, --verbose  Increase the verbosity of messages: 1 for no

Available commands:
  about                Display basic information about your applica
  clear-compiled       Remove the compiled class file
  completion           Dump the shell completion script
  db                   Start a new database CLI session
  docs                 Access the Laravel documentation
  down                 Put the application into maintenance / demo
  env                  Display the current framework environment
  help                 Display help for a command
  inspire              Display an inspiring quote
  list                 List commands
```

Figure 2: Artisan

3.5 Laravel y Visual Studio Code

Como hemos comentado, podemos gestionar muy cómodamente los proyectos Laravel desde el IDE Visual Studio Code. Basta con abrir la carpeta principal del proyecto desde el propio IDE, y eso puede hacerse desde el menú `Archivo > Abrir Carpeta`, o bien arrastrando la carpeta a la zona principal del IDE, e incluso, dependiendo de cómo hayamos hecho la instalación, también podemos hacer clic derecho sobre la carpeta del proyecto en el explorador de archivos, y elegir Abrir con Code en el menú contextual. Una vez abierto, tendremos la estructura del proyecto en la parte izquierda (pestaña de explorador de archivos), y podremos ir editando los archivos en la parte derecha.

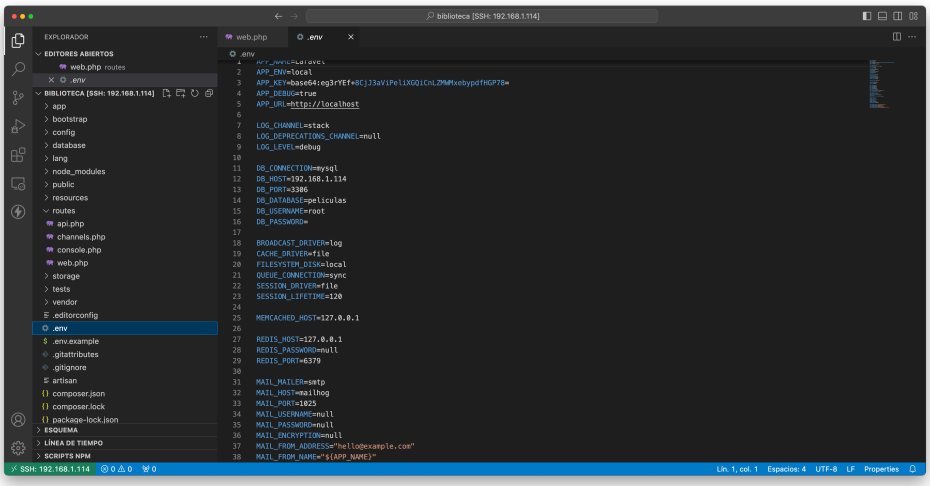


Figure 3: IDE

3.6 Estructura de un proyecto Laravel

Los proyectos Laravel se pueden gestionar abriendo la carpeta directamente desde nuestro IDE (Visual Studio Code). Cuando creamos un proyecto Laravel, se crea una estructura de carpetas y archivos predefinida. Explicamos ahora brevemente en qué consisten las principales carpetas y archivos que se generan.

- `app`: contiene el código fuente de la aplicación. Gran parte de las clases que definamos estarán en esta carpeta. Inicialmente, se incluyen algunas subcarpetas dentro:
  - `Console` : para definir nuestros propios comandos
  - `Exceptions` : para definir nuestras propias excepciones
  - `Http` : contiene los controladores y el middleware
  - `Models` : almacena los modelos de datos de la aplicación (clases que se utilizarán para gestionar los objetos que intervienen en el sistema).
  - `Providers` : contiene los proveedores de servicios de la aplicación, más los que podamos definir nosotros.
  - Además, podemos definir aquí otras carpetas que podamos necesitar. Por ejemplo, en versiones anteriores de Laravel se creaba aquí a mano la carpeta Models para gestionar los modelos de datos.
- `bootstrap` : contiene el archivo app.php , que es el que pone en marcha la aplicación. Además, contiene la carpeta cache , donde se almacenan los archivos ya cargados por Laravel para acelerar su acceso en futuras peticiones.

- `config` : contiene los archivos de configuración de la aplicación, donde se tienen variables de entorno, o si nuestra aplicación está en desarrollo o producción, o los parámetros de conexión a la base de datos, entre otras cosas.
- `database` : almacena los elementos de gestión de la base de datos, tales como generadores de objetos, migraciones, etc.
- **`public`** : contenido visible de la web. Contiene el archivo index.php , punto de entrada de todas las peticiones a la web, y además podemos definir o generar carpetas donde ubicar el contenido estático del cliente (imágenes, hojas de estilo CSS, archivos JavaScript…).
- `resources` : contiene, por un lado, las vistas de nuestra aplicación. Por otro lado, también contiene archivos no compilados de CSS (archivos sass) y JavaScript (archivos sin minimizar u optimizar). Además, también almacena los archivos de traducción, en el caso de que queramos hacer sitios multi idioma.
- `routes` : almacena las rutas de la aplicación, tanto para acceder a contenido web normal (web.php), como para servicios web (api.php), como para comandos y otras opciones. storage : contiene las vistas compiladas, y otros archivos generados por Laravel, como los logs o las sesiones.
- `test` : para almacenar los tests o pruebas sobre los componentes de nuestra aplicación.
- `vendor` : donde se almacenan las dependencias o librerías adicionales que se requieren en nuestro proyecto Laravel. Esta carpeta debería ser ignorada por Git, y regenerada cada vez que se clone el repositorio remoto, para evitar almacenar demasiada información innecesaria. Aunque algunos de los conceptos vistos aquí pueden no estar claros aún (como el concepto de middleware, o los proveedores de servicios), los iremos viendo poco a poco durante el curso.

3.7 Configuración general del proyecto.

De entre la estructura de carpetas de un proyecto Laravel vista anteriormente, echaremos ahora un rápido vistazo a dónde se encuentra la configuración general del proyecto. Por un lado, disponemos de un archivo `.env` en la raíz del proyecto, que básicamente contiene una serie de variables de entorno generales. Por ejemplo, se tiene la variable `APP_NAME` con el nombre que queremos que tenga la aplicación, o un conjunto de variables que utilizaremos más adelante para conectar con la base de datos, entre otras cosas:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=biblioteca
DB_USERNAME=root
DB_PASSWORD=
```

En general, los cambios de configuración es preferible hacerlos en este archivo `.env` , de forma que

en los archivos de la carpeta config accederemos a estas variables de entorno definidas en env. Por ejemplo, podemos definir las propiedades del archivo .env de este modo para el proyecto biblioteca que hemos creado anteriormente.

Por otra parte, la carpeta `config` contiene unos archivos generales de configuración. Iremos viendo algunos de ellos en sesiones posteriores, pero, para empezar, podemos echar un vistazo al archivo `config/app.php`, que contiene parámetros de configuración general de la aplicación. Por ejemplo, podemos modificar el nombre de la aplicación, en la propiedad `name` , aunque, como vemos, el nombre lo coge de la propiedad `APP_NAME` del archivo `.env`.

```
'name' => env('APP_NAME', 'Laravel')
```

Es más habitual modificar los valores del archivo `.env` que los que hay en este archivo de configuración. Pero, en algunos casos, sí convendrá acudir a alguno de los archivos de esta carpeta y modificar la información que contiene.

3.8 Permisos en carpetas del proyecto

Para poder probar nuestro proyecto Laravel, además de configurar y poner en marcha el servidor correspondiente es necesario habilitar permisos de acceso y escritura a ciertas carpetas del proyecto, especialmente en sistemas Linux, si movemos el proyecto a una carpeta con permisos reducidos.

- Carpeta `storage` junto con sus subcarpetas y contenidos. En esta carpeta se compilarán las vistas, se generarán los archivos de log, etc, por lo que conviene que esta carpeta tenga permisos de **escritura**.
- Subcarpeta `bootstrap/cache` , donde se almacenará la caché de los archivos ya cargados. Para habilitar los permisos en estas carpetas bajo Linux o Mac OSX, podemos ejecutar estos comandos desde la raíz del proyecto (la opción -R aplica los permisos de forma recursiva):

```
sudo chmod -R 777 bootstrap/cache
sudo chmod -R 777 storage
sudo chmod -R 777 storage/logs
```

4 Puesta en marcha con el comando artisan

La forma más sencilla de probar nuestro proyecto es a través del comando `artisan` . Nos ubicamos en la carpeta raíz del proyecto (recuerda que esto se puede hacer automáticamente abriendo la carpeta con Visual Studio Code y abriendo su terminal), y ejecutamos este comando:

```
osboxes@osboxes:/opt/lampp/htdocs/biblioteca$ php artisan serve

   INFO  Server running on [http://127.0.0.1:8000].

  Press Ctrl+C to stop the server
```

Para poner en marcha el servidor remoto(nuestro caso):

```
osboxes@osboxes:/opt/lampp/htdocs/proveedores$ php artisan serve --host
    =192.168.1.116

   INFO  Server running on [http://192.168.1.116:8000].

  Press Ctrl+C to stop the server
```

Con la ip particuler de cada equipo…

Conectando a la URL que se nos facilita, veremos la pantalla de inicio de nuestro proyecto, que será algo así (aunque el diseño de la página de inicio puede variar dependiendo de la versión de Laravel que usemos):

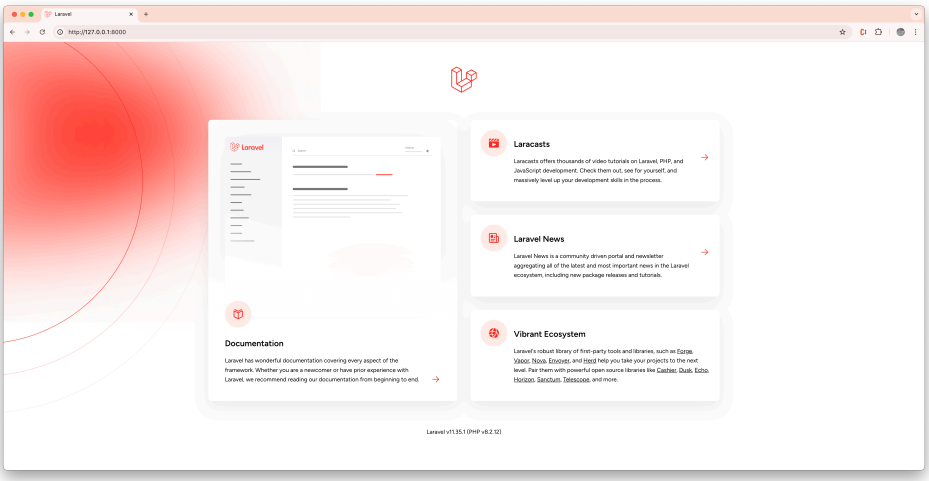


Figure 4: WELCOME



Si nuestra aplicación necesita acceder a una base de datos, además de arrancarla con el comando artisan visto antes, también tendremos que tener en marcha el servidor de bases de datos correspondiente, a través del manager de XAMPP.



En el caso de una puesta en producción real, o si queremos dejar una aplicación correctamente configurada con Apache, hay que realizar una serie de pasos previos, tales como asociarlo a un virtual host (de Apache, en nuestro caso). Estos pasos no los veremos.